instances, it will be easier to set up error conditions. However, with integration tests being run within a deployed environment, it may be more difficult to set up error scenarios.

12.3 Refactoring integration tests to contract tests

Take an integration test like the one in listing 12.2, which has been implemented in Supertest (https://mng.bz/q0vw) to test the movies Express server. The test verifies the get movies response, including headers, status code, and body.

Listing 12.2 Supertest get movies

```
    test("/movies", async () => {
        await request(server)
            .get("/movies")
            .expect("Content-Type", /json/)     #1
            .expect(200)     #2
            .expect((res) => {
                expect(res.body).toMatchObject([])     #3
            })
    })
```

#1 Header
#2 Status code
#3 Response body

The test in listing 12.2 has three `expect` statements, all of which could be verified within a contract test. The `Content-Type` header expects a JSON type, which would be a specific value in the contract. The `OK` status code represents an interaction from the movie's endpoint defining a successful contract interaction. The response body (`res.body`) expects the statement to match the object's properties and values. In this example, the response is an empty array, which could be represented as an array type within the contract. As outlined, all three `expect` statements could be replaced by a contract element. The contract equivalent is shown in listing 12.3. The `uponReceiving` method sets the state of the data required; no data exists in this example. The `withRequest` method sets the request method and path to get all movies. The `willRespondWith` method defines the expected headers, status code, and response body. After the provider expectations have been set, the request is executed with the mock provider URL.